Spatial Logic Puzzles: Tetris – Exploring the Highest Scoring AI Agents

Annie Wang Evan Wang Vivian Song Jonathan Calver Alice Gao

Department of Computer Science, University of Toronto

Overview

Algorithms

Analysis

Results

Spatial Logic Puzzles: Tetris – Exploring the Highest Scoring AI Agents

Annie Wang Evan Wang Vivian Song Jonathan Calver Alice Gao
Department of Computer Science, University of Toronto

Overview

Analysis

Methods

Heuristics

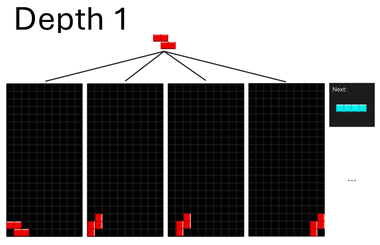
Algorithms

Greedy

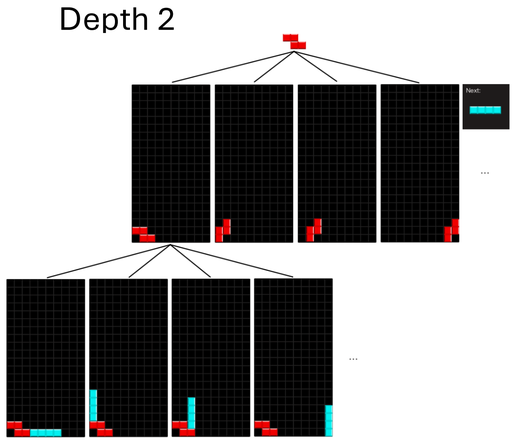
Depth Limited Search (DLS)

Description ajksdhakjsdhgjrwad;ksjdhkldfjhksajdlkfsdhujk

Depth 1



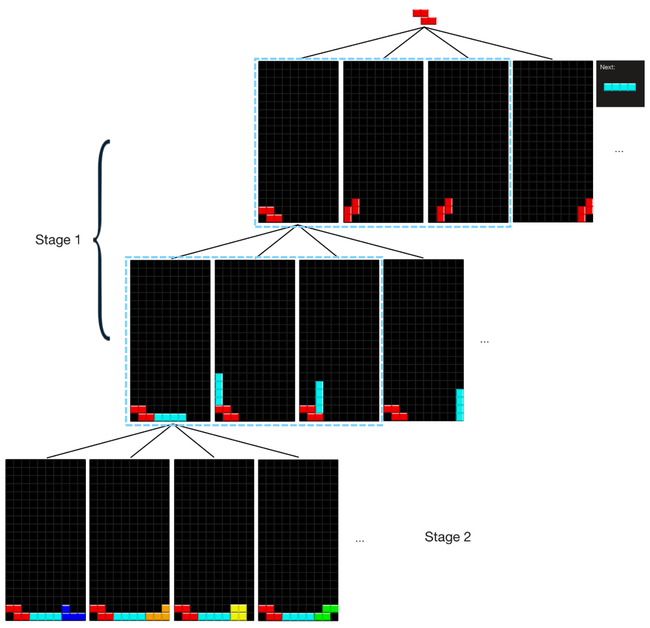
Depth 2



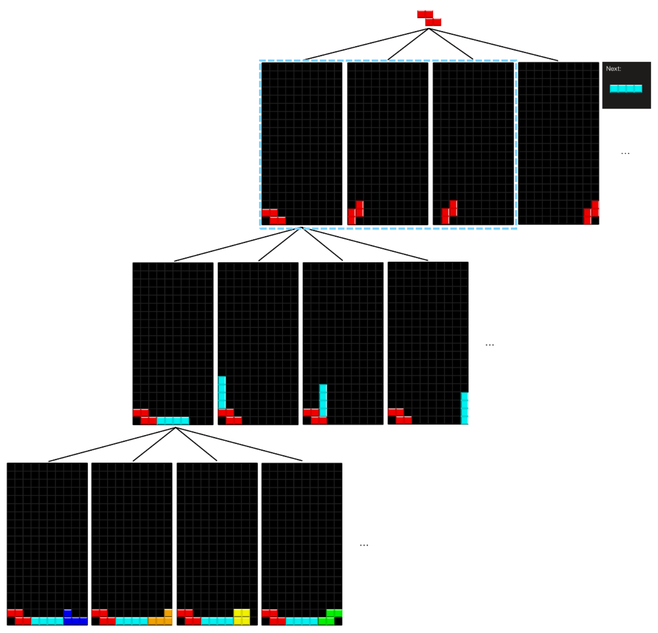
Problem akjsdhakjsdhjkhsdkjahskjdhak jhksadbas

Expectimax

Two Stage Filtering (TSF)



Beam Search(TSF)



Optimizers

Results

Spatial Logic Puzzles: Tetris – Exploring the Highest Scoring AI Agents

Annie Wang Evan Wang Vivian Song Jonathan Calver Alice Gao
Department of Computer Science, University of Toronto

Overview

This project explores different AI agents for the spatial logic game Tetris, as well as various optimization algorithms to compare and evaluate the performance of each agent against each other. Each agent and optimizer was implemented in Python with each agent being independently run, with and without optimizers. This study observes how different

Heuristics

First, we started with 4 main heuristics and weights from Yiyuan Lee[1]
- Aggregate Height: The sum of the heights of every column
- Bumpiness: The sum of absolute height differences between adjacent columns
- Holes: Empty spaces with at least one block above
- Line Clear Utility: Number of lines cleared by the move

After introducing a more sophisticated feature set, the following heuristics were added:
- Column Transitions: How often a cell changes from filled to empty across every column
- Row Transitions: How often a cell changes from filled to empty across every row.
- Maximum Height: Distance from the floor of the board to the highest occupied cell
- Cumulative Wells: Number of empty/sequence of cells that has blocks on both the left & right

Algorithms

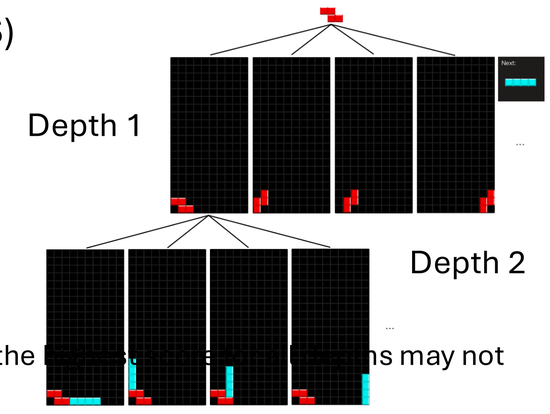
Greedy

Feature	Description
Aggregate Height	Sum of heights of every column
Bumpiness	Sum of absolute height differences between adjacent columns
Holes	Empty spaces with at least one block above
Line Clear Utility	Number of lines cleared by move
Column Transitions	How often a cell changes from filled to empty across every column
Row Transitions	How often a cell changes from filled to empty across every row
Maximum Height	Distance from the floor of board to highest occupied cell
Cumulative Wells	Number of empty cells that has blocks on both left and right

Depth Limited Search (DLS)

DLS performs DFS (LIFO) but only for a specific depth limit. It calculates the score for each path and picks the current piece based on the best path score.

With this implementation, we face a limitation that we can only explore 2 pieces due to original Tetris only providing 1 next piece.



Expectimax

Feature	Description
Aggregate Height	Sum of heights of every column
Holes	Empty spaces with at least one block above
Row Transitions	How often a cell changes from filled to empty across every row
Maximum Height	Distance from the floor of board to highest occupied cell

Now that we are able to explore multiple depths, we face the issue of slow calculations due to too many states, especially with greater depths.

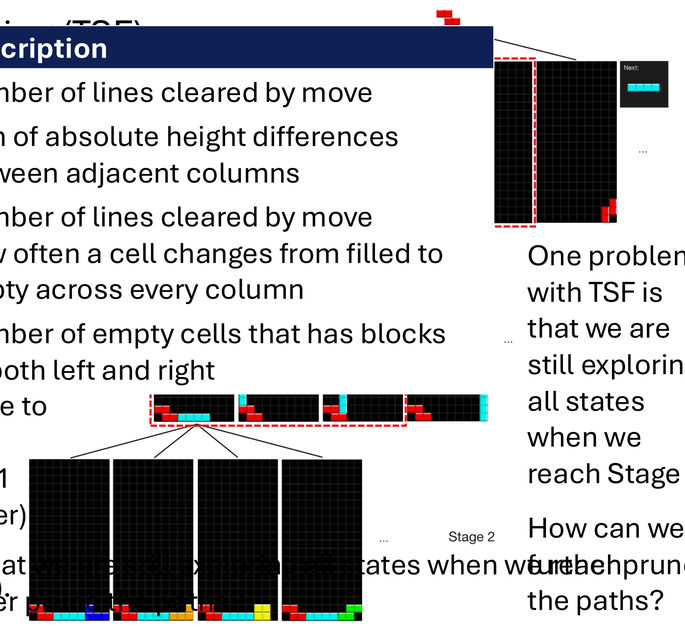
Two-Stage Filtering (TSF)

Feature	Description
Line Clear Utility	Number of lines cleared by move
Bumpiness	Sum of absolute height differences between adjacent columns
Column Transitions	Number of lines cleared by move How often a cell changes from filled to empty across every column
Cumulative Wells	Number of empty cells that has blocks on both left and right

With TSF, we are able to choose different heuristics for stage 1 (simpler and cheaper) and stage 2 (more complex and costly).

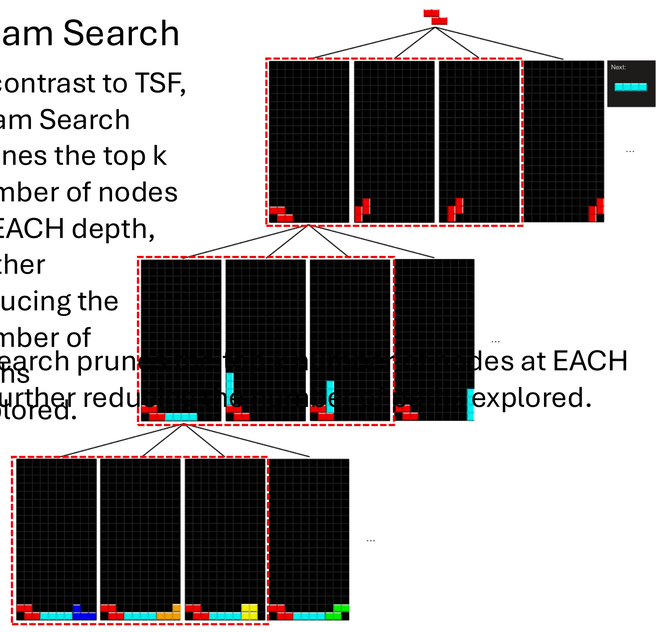
One problem with TSF is that we are still exploring all states when we reach Stage 2.

How can we further prune the paths?



Beam Search

In contrast to TSF, Beam Search prunes the top k number of nodes at EACH depth, further reducing the number of paths explored.



Optimizers

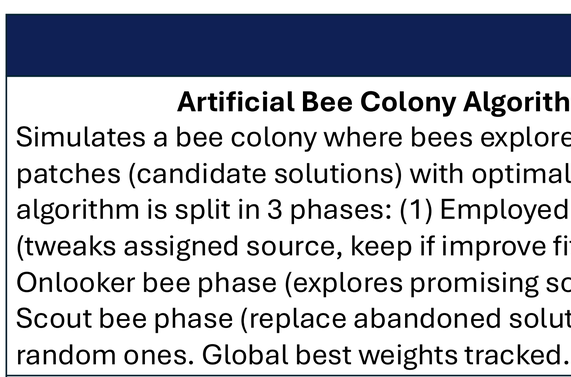
Artificial Bee Colony Algorithm
Simulates a bee colony where bees explore flower patches (candidate solutions) with optimal fitness. The algorithm is split in 3 phases: (1) Employed bee phase (tweaks assigned source, keep if improve fitness) (2) Onlooker bee phase (explores promising solutions) (3) Scout bee phase (replace abandoned solution with random ones. Global best weights tracked.

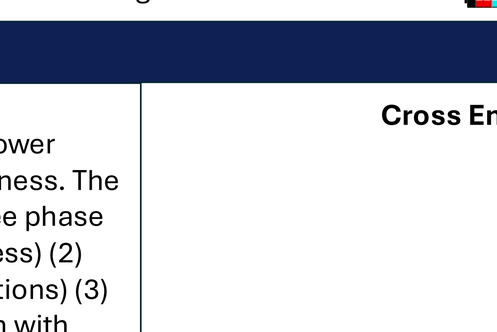
Cross Entropy Algorithm

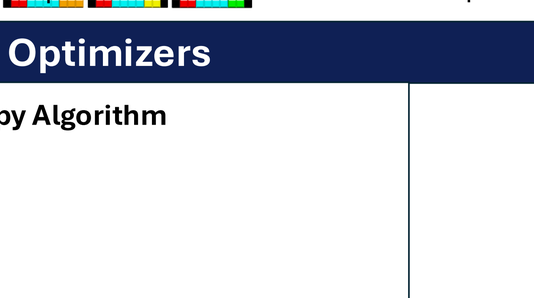
Ant Colony Algorithm
Uses an 8D weight vector

Genetic Algorithm

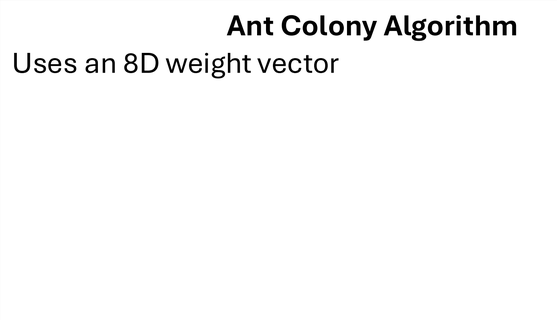
Analysis







Findings ajksdhakjsdhgjrwad;ksjdhkldfjhksajdlkfsdhujk asdjaksdhkasasdkhakjsdhkasdhasdkjhasdhaks







Findings ajksdhakjsdhgjrwad;ksjdhkldfjhksajdlkfsdhujk asdjaksdhkasasdkhakjsdhkasdhasdkjhasdhaks

Results

[1] Lee, Y (2013)

Spatial Logic Puzzles: Tetris – Exploring Strategies for the Highest Scoring Agents

Annie Wang Evan Wang Vivian Song Jonathan Calver Alice Gao

Department of Computer Science, University of Toronto

Overview

Rules

Environment: The game space is defined as a grid of 10 x 20 cells. The state of the system is updated as Tetriminos occupy specific coordinates.
Tetriminos: There are seven distinct shapes, each composed of four square units. These are governed by the following constraints:

- **Generation:** Pieces are generated via a random generator to ensure a uniform distribution
- **Rotation:** Pieces rotate around a specific pivot point based on the **Super Rotation System (SRS)**
- **Collision:** A piece is "locked" when its downward movement is obstructed by the floor or an occupied cell

Operational Mechanics

- **Line Clear Logic:** If all 10 cells in a horizontal row y are occupied, the row is deleted. All rows y' > y are then translated downward by one unit
- **Terminal State (Top-Out):** The process terminates when a new Tetrimino is unable to spawn in the "Buffer Zone" due to existing occupied cells

Scoring Algorithms (Quantitative Metrics)

- **Back-to-Back Multipliers:** Awarded for consecutive high-value clears (Tetris) without intermittent low-value clears

Algorithms

One-step Lookahead with State-Space Search

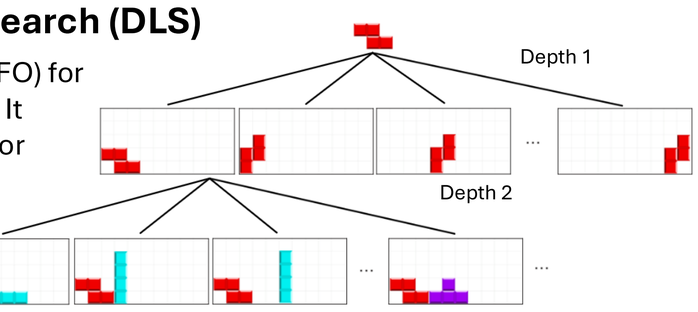
Calculates a score S for every possible piece placement of the current piece on the board:
$S = w_1 f_1 + \ldots + w_n f_n$ where f_i is the i-th feature extracted from the board and w_i is the weight assigned to that feature

Feature Set:

- **Aggregate Height**: Sum of heights of every column
- **Bumpiness**: Sum of absolute height differences between adjacent columns
- **Holes**: Empty spaces with at least one block above
- **Line Clear Utility**: Number of lines cleared by move
- **Column Transitions**: How often a cell changes from filled to empty across every column
- **Row Transitions**: How often a cell changes from filled to empty across every row
- **Maximum Height**: Distance from the floor of board to highest occupied cell
- **Cumulative Wells**: Number of empty cells that has blocks on both left and right

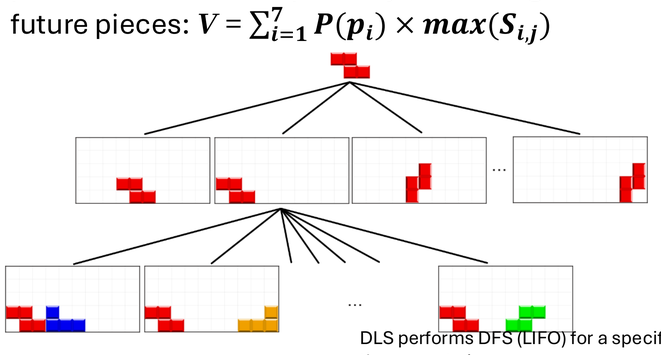
Depth Limited Search (DLS)

DLS performs DFS (LIFO) for a specific depth limit. It calculates the score for each path and picks the current piece based on the best path score.

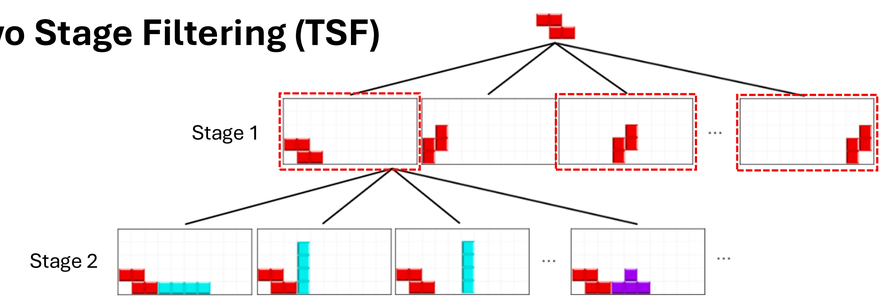


Depth-Limited Expectimax

Calculates an expected score V for every possible placement of current piece by simulating all future pieces: $V = \sum_{i=1}^{7} P(p_i) \times max(S_{i,j})$

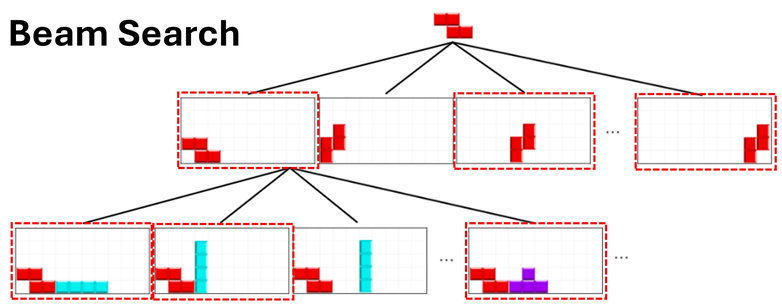


Two Stage Filtering (TSF)



Search optimization technique that prunes the state-space.
TSF separates depths into 2 stages:
Stage 1: Chooses k highest scoring paths using a simplified heuristic set
Stage 2: Only for chosen nodes, perform DLS with full heuristic set for remaining depth

Beam Search



Heuristic search algorithm that expands only the most promising k nodes at each depth

Optimizers

Artificial Bee Colony Algorithm

Simulates a bee colony where bees explore flower patches (candidate solutions) with optimal fitness. The algorithm is split in 3 phases:

1. Employed bee phase (tweaks assigned source, keep if improve fitness)
2. Onlooker bee phase (explores promising solutions)
3. (3) Scout bee phase (replace abandoned solution with random ones.

Ant Colony Algorithm

Uses an 8D weight vector; weights are represented as discrete bins so an ant = one bin choice per feature. It evaluates each sampled weight set using the agent, then eliminates old pheromones and reinforces the value choices used by the best performing ants. The best-scoring weight vector found is kept.

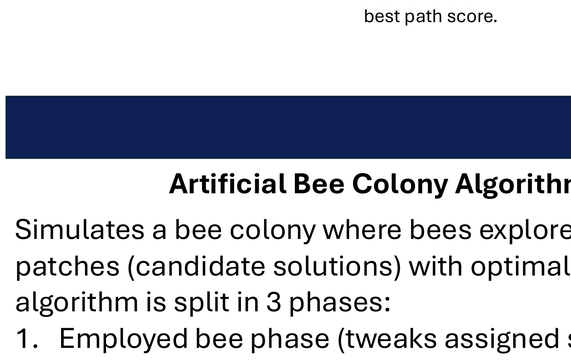
Cross Entropy Algorithm

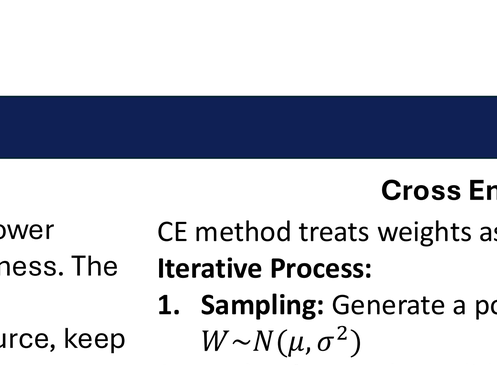
CE method treats weights as a probability distribution and evolves them over multiple generations
Iterative Process:

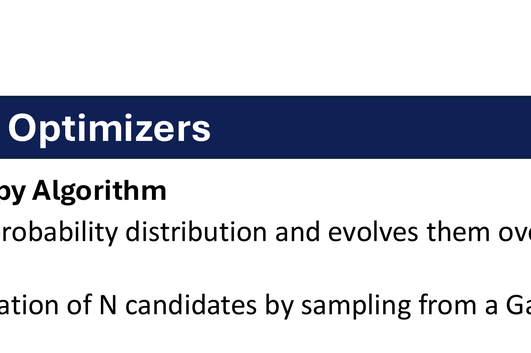
1. **Sampling:** Generate a population of N candidates by sampling from a Gaussian distribution: $W \sim N(\mu, \sigma^2)$
2. **Evaluation:** Each candidate plays a set of games. The final score is recorded as its "fitness"
3. **Selection:** Identify the top $X\%$ of candidates (the "Elite" set)
4. **Updating:** Shift the distribution's mean and variance toward the Elite set:
   - $\mu_{new} = mean(W_{elite})$
   - $\sigma_{new} = std(W_{elite})$

Genetic Algorithm

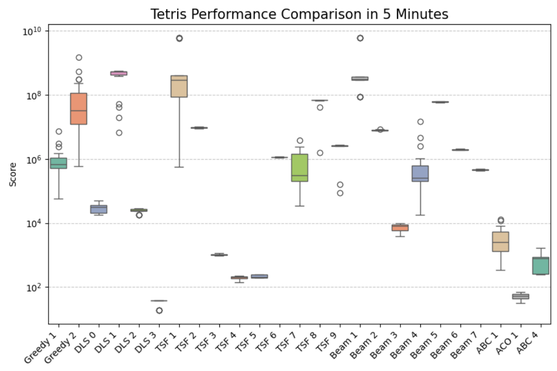
Analysis

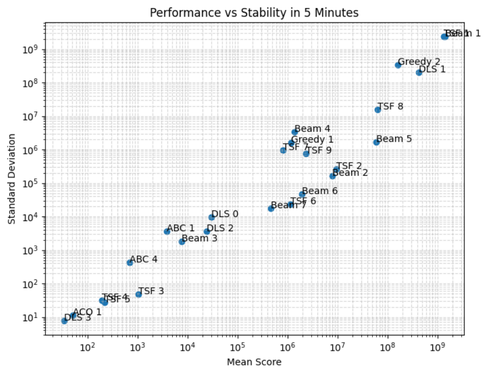


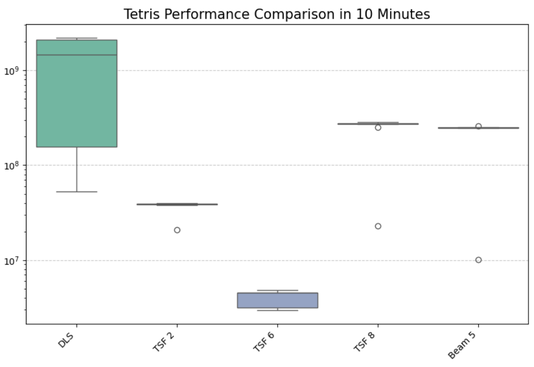




Findings ajksdhakjsdhgjrwad;ksjdhkldfjhksajdlkfsdhujk asdjaksdhkasasdkhakjsdhkasdhasdkjhasdhaks







Findings ajksdhakjsdhgjrwad;ksjdhkldfjhksajdlkfsdhujk asdjaksdhkasasdkhakjsdhkasdhasdkjhasdhaks

Results

[1] Lee, Y (2013)

Spatial Logic Puzzles: Tetris – Exploring Strategies for the Highest Scoring Agents

Annie Wang Evan Wang Vivian Song Jonathan Calver Alice Gao

Department of Computer Science, University of Toronto

Overview

Objective: This study evaluates the performance of various Python-based AI agents and optimization algorithms within the spatial logic environment of Tetris.

- Compares trade-offs between heuristic agents
- Feature weights are tuned to evaluate how different algorithms affect agent performance

Rules

Environment: Game space is defined as a grid of 10 x 20 cells. The state of the system is updated as Tetrominos occupy specific coordinates.
Tetrominos: There are seven distinct shapes, each composed of four square units. These are governed by the following constraints:

- **Generation:** Pieces are generated via a random generator to ensure a uniform distribution
- **Rotation:** Pieces rotate around a specific pivot point based on the **Super Rotation System (SRS)**

Operational Mechanics

- **Line Clear Logic:** If all 10 cells in a horizontal row y are occupied, the row is deleted. All rows y' > y are then translated downward by one unit
- **Terminal State (Top-Out):** The process terminates when a new Tetromino is unable to spawn in the "Buffer Zone" due to existing occupied cells

Back-to-Back Multipliers: Awarded for consecutive high-value clears (Tetris) without intermittent low-value clears

Algorithms

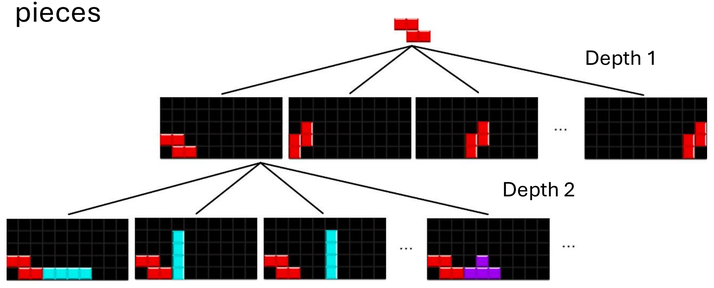
One-step Lookahead with State-Space Search

Calculates a score S for every possible piece placement of the current piece on the board: $S = w_1 f_1 + \ldots + w_n f_n$, where f_i is the i-th feature extracted from the board and w_i is the weight assigned to that feature

Feature	Description	Feature	Description
Aggregate Height	Sum of heights of every column	**Line Clear Utility**	Number of lines cleared by move
Holes	Empty spaces with at least one block above	**Bumpiness**	Sum of absolute height differences between adjacent columns
Row Transitions	How often a cell changes from filled to empty across every row	**Column Transitions**	Number of lines cleared by move How often a cell changes from filled to empty across every column
Maximum Height	Distance from the floor of board to highest occupied cell	**Cumulative Wells**	Number of empty cells that has blocks on both left and right

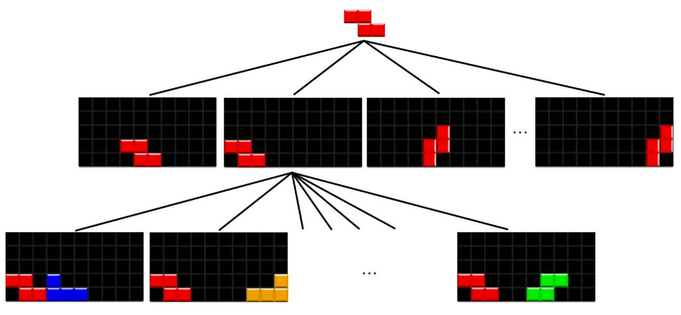
Depth Limited Search (DLS)

Explores a decision tree up to fixed maximum depth d. For a depth of d, DLS simulates a sequence of d future pieces



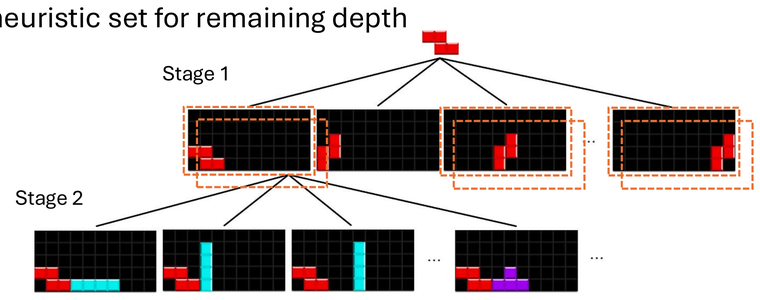
Depth-Limited Expectimax

Calculates an expected score V for every possible placement of current piece by simulating all future pieces: $V = \sum_{i=1}^{7} P(p_i) \times max(S_{i,j})$



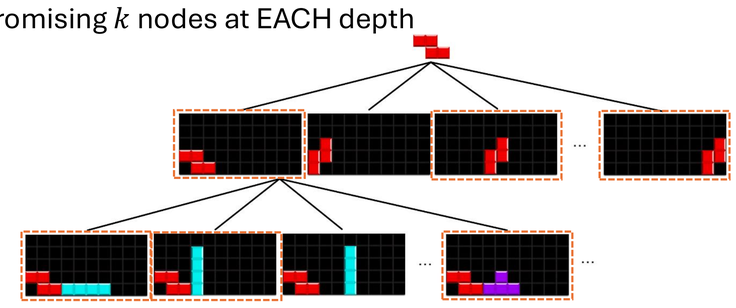
Two Stage Filtering (TSF)

Search optimization technique that prunes the state-space. TSF separates depths into 2 stages:
Stage 1: Chooses k highest scoring paths using a simplified heuristic set
Stage 2: Only for chosen nodes, perform DLS with full heuristic set for remaining depth



Beam Search

Heuristic search algorithm that expands only the most promising k nodes at EACH depth



Optimizers

Cross Entropy Method

CE method treats weights as a probability distribution and evolves them over multiple generations

Iterative Process:

1. **Sampling:** Generate a population of N candidates by sampling from a Gaussian distribution: $W \sim N(\mu, \sigma^2)$
2. **Evaluation:** Each candidate plays a set of games. The final score is recorded as its "fitness"
3. **Selection:** Identify top $X\%$ of candidates ("Elite" set)
4. **Updating:** Shift distribution's mean and variance toward Elite set:
   - $\mu_{new} = mean(W_{elite})$
   - $\sigma_{new} = std(W_{elite})$

Genetic Algorithm

GA maintains a population of discrete agents, using Tournament Selection and Uniform Crossover to breed high-performing strategies.

1. **Selection:** Agents that achieve higher game scores are more likely to be chosen as "parents"
2. **Crossover:** Pairs of parents swap segments of their weight values to produce "offspring," combining successful traits from both
3. **Mutation:** Random noise is injected into weights to maintain genetic diversity and prevent population from getting stuck in local optima

Artificial Bee Colony Algorithm

Simulates a bee colony where bees explore flower patches (candidate solutions) with optimal fitness. The algorithm is split in 3 phases:

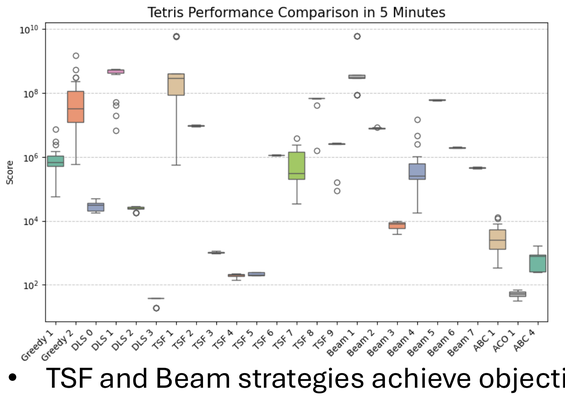
1. **Employed bee phase** (tweaks assigned source, keep if improve fitness)
2. **Onlooker bee phase** (explores promising solutions)
3. **Scout bee phase** (replace abandoned solution with random ones.

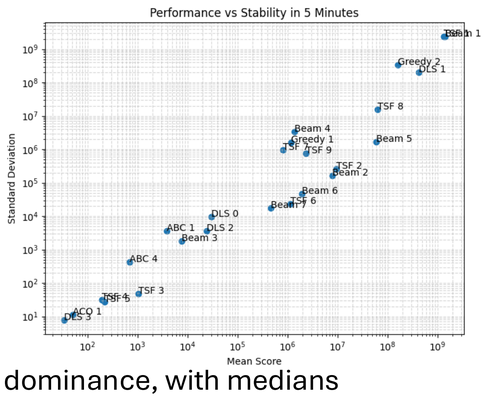
Ant Colony Algorithm

ACO builds candidate solutions, rewards strongest, and reinforces best choices through the following steps:

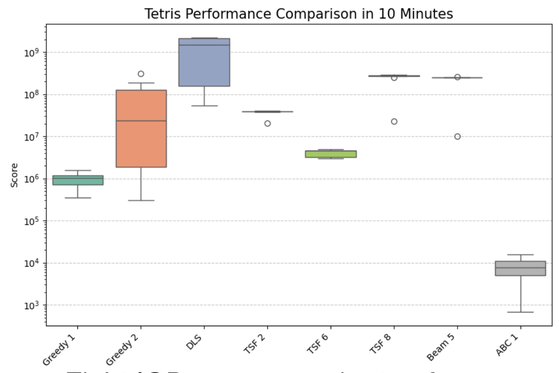
1. **Ant construction** (sample a candidate from pheromone)
2. **Evaluation** (score the candidate)
3. **Pheromone update** (reinforce good choices, evaporate old trails)

Analysis

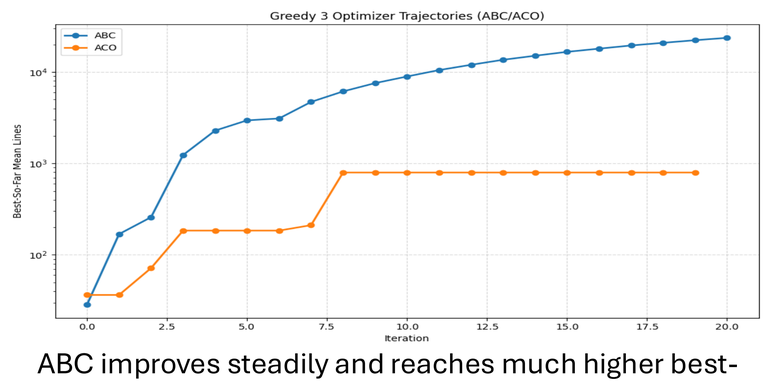




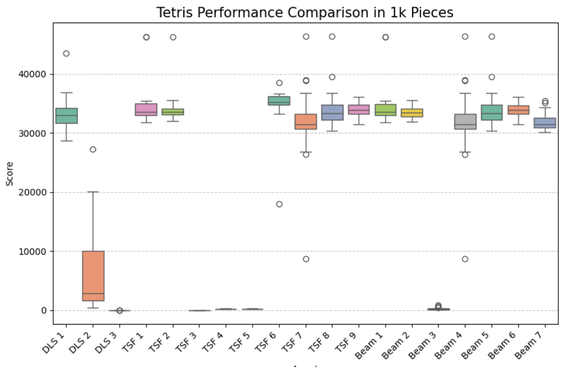
- TSF and Beam strategies achieve objective dominance, with medians exceeding the entire interquartile range of baseline agents
- Tight boxes for TSF indicate it is a highly stable strategy

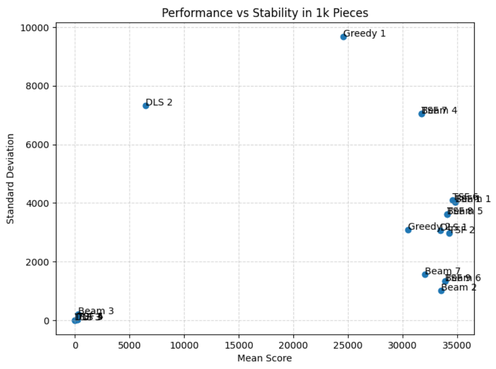


- Tight IQRs across variants of lookahead strategies demonstrate high "RNG-proof" consistency

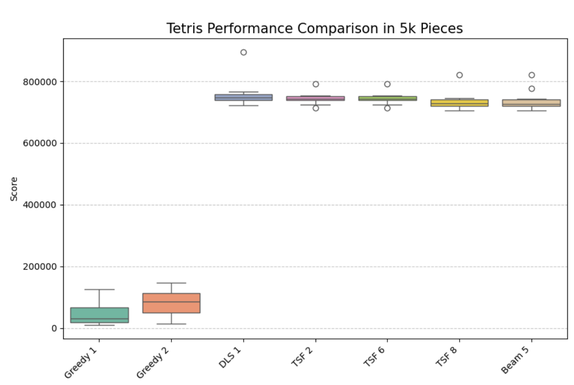


ABC improves steadily and reaches much higher best-so-far mean lines than ACO on greedy_3. ACO rises early but plateaus quickly.

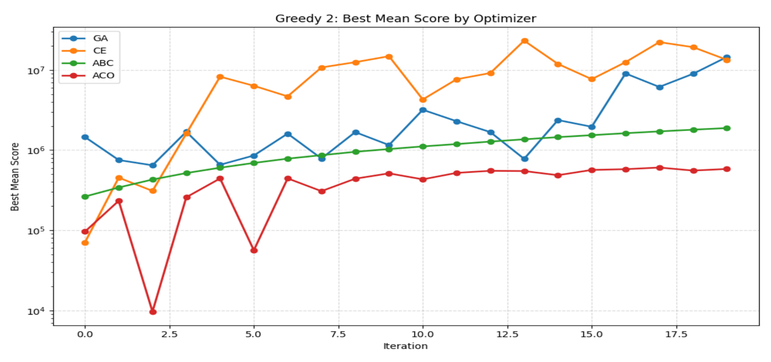




- Most agents exhibit high precision and stability compared to baseline DLS
- High-performing variants converge at 35k points, indicating they successfully reached the 1k-piece cap rather than hitting an algorithmic limit and can continue
- Some agents fail to complete computation before pieces reach the board



- Large increase in scores indicates lookahead is more critical than heuristic tuning for long-term survival

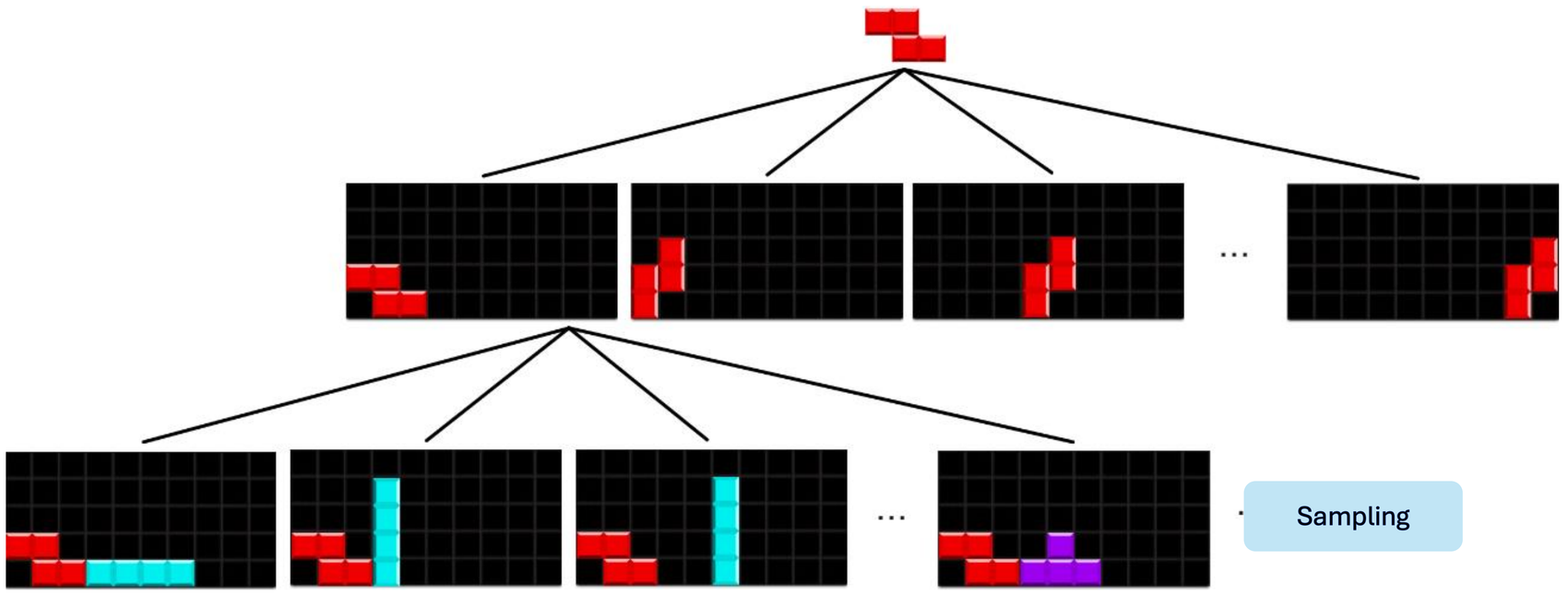


CE reaches the highest best mean scores overall, while GA also performs strongly with larger fluctuations. ABC improves more gradually, and ACO remains the weakest across most iterations.

Algorithms	Pros	Cons
One-step Lookahead	• Extremely fast • Easy to tune weights	• Short-sighted • Often creates wells because of no lookahead
DLS	• Strategic placement • Higher ceiling	• Computation cost • Diminishing returns
TSF	• Ignores obviously bad paths, increasing efficiency	• Pruning risk (might discard good move) • Harder to balance threshold of which moves pass
Expectimax	• Probabilistic robustness	• Computationally heavy
Beam	• Controlled complexity • Deep reach without exponential explosion	• No optimality guarantee

References & Related Work

1. Christophe Thiery, Bruno Scherrer. Building Controllers for Tetris. International Computer Games Association Journal, 2009, 32, pp.3-11. ffinria-00418954f
2. Dunham, M., & Alves, A. (n.d.). Tetris game-playing agents in python. https://www.cs.uml.edu/ecg/uploads/AIfall14/dunham_alves_tetris_game_playing.pdf
3. Gilliland, T., & Zhang, D. (n.d.). Parallel greedy tetris solver. https://www.cs.columbia.edu/~sedwards/classes/2020/4995-fall/reports/Tetris.pdf
4. Jaivigneshvenugopal. (n.d.). *Jaivigneshvenugopal/AI-Tetris-Agent: A computer program that plays the game Tetris without any human aid*. GitHub. https://github.com/jaivigneshvenugopal/ai-tetris-agent
5. Lee, Y. (2017, June 14). *Tetris AI – the (near) perfect bot*. Code My Road. https://codemyroad.wordpress.com/2013/04/14/tetris-ai-the-near-perfect-player/
6. Naing, T., Truong, A., & Zeng, O. (n.d.). Tetris AI. https://thawsitt.me/files/Tetris_AI_CS221_final_paper.pdf



Sampling

Sampling

Generate a population of candidates by sampling from a Gaussian distribution: $W \sim N(\mu, \sigma^2)$

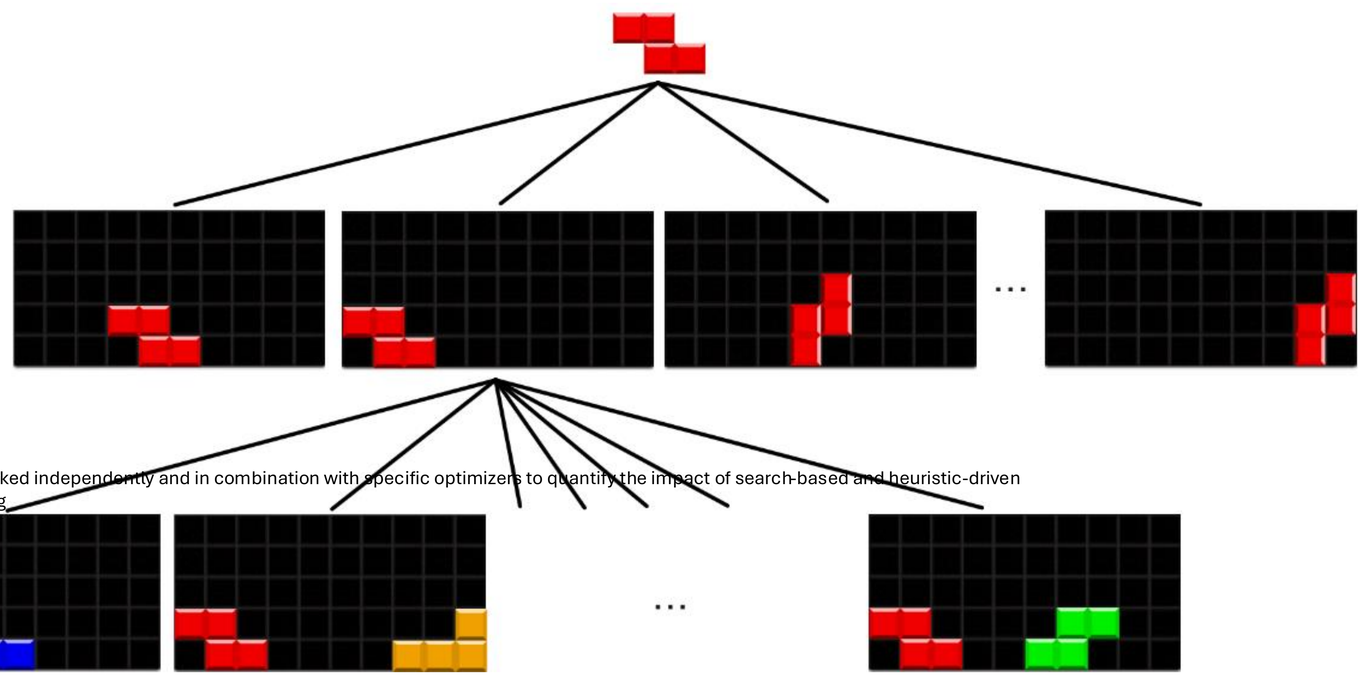
Each candidate plays a set of games. The final score is recorded as its "fitness"

Identify the top $X\%$ of candidates (the "Elite" set)

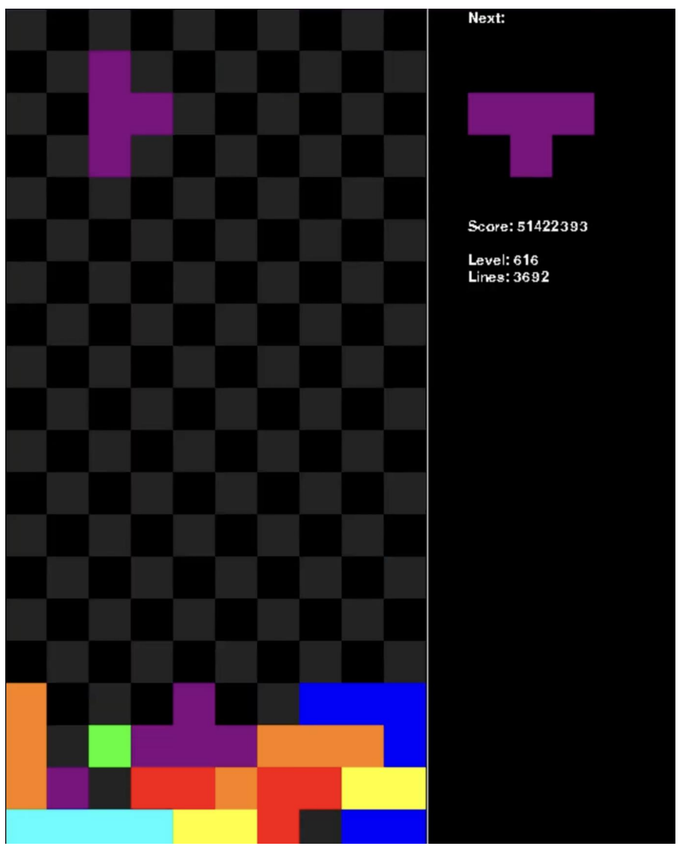
Shift the distribution's mean and variance toward the Elite set:

- $\mu_{new} = mean(W_{elite})$
- $\sigma_{new} = std(W_{elite})$

1. **Selection:** Agents that achieve higher game scores are more likely to be chosen as "parents"
2. **Crossover:** Pairs of parents swap segments of their weight values to produce "offspring," combining successful traits from both
3. **Mutation:** Random noise is injected into the weights to maintain genetic diversity and prevent population from getting stuck in local optima



Methodology: Each agent was benchmarked independently and in combination with specific optimizers to quantify the impact of search-based and heuristic-driven strategies on game endurance and scoring



Simulates a bee colony where bees explore flower patches (candidate solutions) with optimal fitness. The algorithm is split in 3 phases:

1. **Employed bee phase** (tweaks assigned source, keep if improve fitness)
2. **Onlooker bee phase** (explores promising solutions)
3. **Scout bee phase** (replace abandoned solution with random ones.